

VDICL

Authoring Guide

VFlow Decision Rule Language

Complete Reference for Rule Pack Authoring
Expressions · Schema · Functions · Hit Policies · Runtime

Version 1.0 · 2026

Table of Contents

- 1. Overview 3
- 2. Authoring Style 4
- 3. Rule Pack YAML Structure 5
- 4. Schema and Fields 7
- 5. Expressions and Operators 9
- 6. Literals and Values 11
- 7. Built-in Functions 13
- 8. Actions and Outputs 20
- 9. Hit Policies 22
- 10. DMN Unary Tests 24
- 11. Validation and Runtime 25
- 12. Complete Example 27

1. Overview

VDICL (VFlow Decision and Conditions Language) is the rule-authoring language used inside VFlow decision engine rule packs. A rule pack is a YAML document that defines decision logic as an ordered list of condition–action rules. The runtime compiles the pack and evaluates it against an input context to produce a structured decision result.

VDICL is designed with two layers:

- YAML container — defines pack structure, metadata, schema, and rule list.
- VDICL expressions — appear inside selected YAML fields (when, expr) and are parsed by the compiler, not treated as ordinary strings.

Key Design Principles

- Rule packs are pure declarations. No side effects occur during condition evaluation.
- All field references are schema-bound at compile time. Unknown fields are errors, not warnings.
- Output is deterministic given the same input and hit policy.
- Business rules (VDICL packs) are separated from workflow orchestration (VWFD YAML).

□ *This guide covers the public authoring surface. The implementation source of truth is the VFlow VDICL compiler, runtime, and schema model.*

2. Authoring Style

VDICL authoring uses YAML as a container and VDICL expressions as values inside selected YAML fields. Keep those two layers separate.

2.1 Canonical YAML Style

Write description, when, path, and expr as plain YAML scalars — no surrounding quotes:

```
description: Applicant must be at least 21
when: applicant.age >= 21
path: decision
expr: applicant.monthlyIncome * 12
```

❑ *Avoid wrapping the whole field value in YAML quotes. Quoted scalars prevent LSP, diagnostics, highlighting, and AI editing tools from recognising expressions.*

2.2 String Literals

Use double quotes only where the expression needs a string literal:

```
when: tier == "GOLD" AND region == "ID"
value: "APPROVE"
```

2.3 Logical Operators

Use AND, OR, and NOT as canonical forms. The runtime also accepts && and || but authored packs should prefer the keyword forms for clarity.

```
when: active == true AND score >= 700
when: tier == "GOLD" OR tier == "PLATINUM"
when: NOT blocked
```

2.4 Paths

Paths are dotted identifiers. Do not wrap them in quotes — authoring tools cannot treat a quoted path as navigable:

```
when: applicant.creditScore >= 650
path: approvedLimit
```

2.5 Computed vs. Literal Output

Use SET_FIELD for literal string output and EVAL_SET_FIELD for computed output:

```
# Literal output
- kind: SET_FIELD
  path: decision
  value: "APPROVE"

# Computed output
- kind: EVAL_SET_FIELD
  path: approvedLimit
  expr: applicant.monthlyIncome * 12
```

❑ *Never put a computed expression under value. The value field is a literal string in the rule model.*

3. Rule Pack YAML Structure

A VDICL rule pack is a YAML document with rule metadata, schema information, hit policy, and an ordered list of rules.

3.1 Minimal Rule Pack

```
version: 1
decision_id: eligibility
decision_name: Eligibility Decision
hit_policy: UNIQUE
schema:
  inputs:
    - name: tier
      type: string
      expression: applicant.tier
    - name: approvedLimit
      type: number
      expression: applicant.approvedLimit
  outputs:
    - name: decision
      type: string
rules:
  - id: r_gold_high_limit
    description: Approve GOLD tier with approved limit above 10000
    when: tier == "GOLD" AND approvedLimit > 10000
    then:
      - kind: SET_FIELD
        path: decision
        value: "APPROVE"
  - id: r_default_review
    description: Review all cases not matched by a specific rule
    when: true
    then:
      - kind: SET_FIELD
        path: decision
        value: "REVIEW"
```

3.2 Top-Level Fields

Field	Required	Description
version	Recommended	Authoring document version.
decision_id	Recommended	Stable identifier for this decision.
decision_name	Recommended	Human-readable decision name.
metadata	Optional	Deployment or ownership metadata.
schema_ref	Optional	External schema reference path.
schema	Recommended	Input/output shape used by tooling.
hit_policy	Optional	Rule matching policy. Default: FIRST.
output_conflict_policy	Optional	LWW or STRICT. Default: LWW.

Field	Required	Description
aggregate_fn	Optional	Aggregation for COLLECT. Default: LIST.
functions	Optional	User functions for Starlark-backed packs.
rules	Required	Ordered list of rule objects.

3.3 Rule Fields

Field	Required	Description
id	Yes	Stable rule identifier. Must be unique within the pack.
priority	No	Signed integer used by PRIORITY hit policy. Default: 0.
enabled	No	Disabled rules are skipped entirely. Default: true.
description	Yes	Human-readable description of the rule intent.
when	Yes	VDICL expression compiled and evaluated at runtime.
then	Yes	Ordered list of action objects executed when when is true.

3.4 Rule Order

Document order matters for FIRST, RULE_ORDER, and deterministic fallback. Place specific rules before fallback rules:

```
rules:
- id: r_specific_case
  description: Approve high confidence case
  when: score >= 800
  then:
    - kind: SET_FIELD
      path: decision
      value: "APPROVE"
- id: r_fallback
  description: Review all remaining cases
  when: true
  then:
    - kind: SET_FIELD
      path: decision
      value: "REVIEW"
```

4. Schema and Fields

VDICL expressions are checked against a field dictionary at compile time. The dictionary defines available input paths, scalar kinds, and nullability.

4.1 Authoring Schema

```
schema:
  inputs:
    - name: age
      type: number
      expression: applicant.age
    - name: tier
      type: string
      expression: applicant.tier
    - name: verified
      type: boolean
      expression: applicant.verified
  outputs:
    - name: decision
      type: string
```

4.2 Scalar Kinds

Scalar Kind	Authoring Type	Description
NULL	null	Null value.
BOOL	boolean	Boolean true/false.
I32, I64	number	Signed integer (32-bit or 64-bit).
U32, U64	number	Unsigned integer (32-bit or 64-bit).
F64	number	IEEE 754 double-precision floating-point.
DECIMAL128	decimal	High-precision decimal number.
SYM	string	Canonicalized symbolic text.
BYTES	string	Byte-preserved string data.
DATE_DAYS	date	Date represented as day count.
DATETIME_MICROS	datetime	Datetime in microseconds since epoch.
TIME_MICROS	time	Time of day in microseconds.
DAY_TIME_DURATION	duration	Day-time duration value.
YEAR_MONTH_DURATION	duration	Year-month duration value.
JSON	json	Arbitrary JSON payload.
PATH	path	Path value.
LIST	list	Ordered list value.
OBJECT	object	Context or object value.

4.3 Field Paths

Use dotted notation for nested input access:

```
when: applicant.creditScore >= 650  
when: applicant.address.country == "ID"
```

Field paths must resolve to schema fields. Unknown paths are compile errors.

4.4 Nullability

Nullable fields may evaluate to null. Use explicit null checks where business logic depends on presence:

```
when: applicant.npwp IS NOT NULL AND applicant.npwp != ""
```

Use COALESCE for safe defaulting:

```
when: COALESCE(applicant.monthlyIncome, 0) >= 5000000
```


5. Expressions and Operators

The `when` field and `EVAL_SET_FIELD.expr` contain VDICL expressions, parsed by the compiler as expression grammar — not as plain YAML strings.

5.1 Examples

```
when: age >= 21
when: tier == "GOLD" AND approvedLimit > 10000
when: applicant.country IN {"ID", "SG", "MY"}
when: applicant.npwp IS NOT NULL
```

5.2 Operator Precedence

From highest binding to lowest:

Level	Operators and Forms
Atom	literals, paths, function calls, grouped expressions
Unary	NOT, unary -
Power	**
Multiplicative	*, /, %
Additive	+, -
Comparison	==, !=, <>, <, <=, >, >=, IN, NOT IN
Suffix predicates	IS NULL, IS NOT NULL, IS BLANK, IS EMPTY
Range sugar	BETWEEN min AND max
Type check	instance of number/string/boolean/null/list/context/object
Logical AND	AND
Logical OR	OR

Use parentheses for explicit grouping when clarity demands it:

```
when: (tier == "GOLD" OR tier == "PLATINUM") AND approvedLimit > 10000
```

5.3 Comparison Operators

```
when: amount == 1000000
when: amount != 0
when: amount <> 0      # alias for !=
when: age >= 21
when: riskScore < 40
```

5.4 Membership (IN / NOT IN)

```
when: tier IN {"GOLD", "PLATINUM"}
when: country NOT IN {"IR", "KP"}
```

5.5 Null and Blank Checks

```
when: email IS NOT NULL
when: phone IS BLANK
when: attachments IS EMPTY
```

5.6 Range Sugar

```
when: age BETWEEN 21 AND 65
```

5.7 Conditional Expression

FEEL-style if-then-else is supported inside expr fields:

```
expr: if score >= 700 then "APPROVE" else "REVIEW"
```

5.8 Iterators and List Expressions

```
when: exists item in riskFlags satisfies item == "FRAUD"
when: every item in requiredChecks satisfies item == true
expr: for item in scores return item * 2
expr: transactions[item.amount > 1000000] # list filter
```

6. Literals and Values

6.1 Null and Booleans

Use lowercase true, false, and null in all expressions:

```
when: verified == true
when: blocked == false
when: npwp IS NULL
```

6.2 Numbers

Integers, decimals, negative numbers, and exponent notation are all valid:

```
when: amount > 1000000
when: ratio >= 0.75
when: score >= .5
when: exposure < 1.25e6
when: margin >= 12.50m    # m/M suffix for decimal literal
```

6.3 Strings

Use double quotes for string literals in expressions:

```
when: tier == "GOLD"
when: country == "ID"
```

Use the raw byte form only for cases that require exact byte matching:

```
when: MATCHES_REGEX(code, b'^[A-Z]{3}[0-9]{4}$')
```

6.4 Sets

Sets are used with IN and NOT IN operators:

```
when: tier IN {"GOLD", "PLATINUM"}
```

6.5 Lists

List values are primarily produced by input JSON and list functions. Use list builtins (COUNT, SUM, SUBLIST, APPEND, DISTINCT) to operate on them:

```
when: COUNT(requiredDocuments) >= 3
expr: APPEND(flags, "MANUAL_REVIEW")
```

6.6 Contexts

Context/object values are produced by context functions and JSON input:

```
expr: CONTEXT("decision", decision, "reason", reasonCode)
expr: GET_VALUE(applicantContext, "tier")
```

6.7 Literal vs. Computed Output

SET_FIELD.value is a literal string field — never an expression:

```
- kind: SET_FIELD
  path: decision
  value: "APPROVE"
```

For numeric, boolean, or computed output always use EVAL_SET_FIELD:

```
- kind: EVAL_SET_FIELD  
  path: approvedLimit  
  expr: monthlyIncome * 12
```

Built-in Functions

VDICL has a built-in function registry. Function names are ordinary calls in expressions. Do not use names beginning with `__` — those are internal translator helpers.

```
when: COALESCE(monthlyIncome, 0) >= 5000000
when: STARTS_WITH(accountNumber, "ID")
expr: ROUND(monthlyIncome * 0.35, 0)
```

7.1 General

Function	Description
<code>COALESCE(value, ...)</code>	Returns the first non-null value in the argument list.
<code>IFNULL(value, fallback)</code>	Returns fallback when value is null.
<code>UUID()</code>	Returns a new UUID string at runtime.

7.2 String Functions

Function	Description
<code>UPPER(text)</code>	Convert text to uppercase.
<code>LOWER(text)</code>	Convert text to lowercase.
<code>LENGTH(text) / LEN(text)</code>	Return character count.
<code>TRIM(text)</code>	Strip leading and trailing whitespace.
<code>SUBSTR(text, start, len)</code>	Extract substring.
<code>CONCAT(value, ...)</code>	Concatenate values as strings.
<code>CONTAINS(text, needle)</code>	True when text contains needle.
<code>STARTS_WITH(text, prefix)</code>	True when text begins with prefix.
<code>ENDS_WITH(text, suffix)</code>	True when text ends with suffix.
<code>MATCHES_REGEX(text, pattern)</code>	True when text matches regex pattern.
<code>REPLACE(text, from, to)</code>	Replace first occurrence.
<code>REPLACE_FLAGS(text, pat, repl, flags)</code>	Regex replace with flags.
<code>SPLIT(text, separator)</code>	Split text into a list.
<code>STRING(value)</code>	Convert value to string.
<code>STRING_JOIN(list, separator)</code>	Join list items with separator.
<code>SUBSTRING_BEFORE(text, marker)</code>	Portion of text before marker.
<code>SUBSTRING_AFTER(text, marker)</code>	Portion of text after marker.
<code>TO_BASE64(value)</code>	Base64-encode value.

Function	Description
<code>FROM_BASE64 (value)</code>	Base64-decode value.
<code>TO_HEX (value)</code>	Hex-encode value.
<code>NUMBER_FROM_STRING (text)</code>	Parse numeric text to number.

7.3 Numeric Functions

Function	Description
<code>ABS (number)</code>	Absolute value.
<code>ROUND (number, scale)</code>	Round to scale decimal places.
<code>ROUND_UP (number, scale)</code>	Round away from zero.
<code>ROUND_DOWN (number, scale)</code>	Round toward zero (truncate).
<code>ROUND_HALF_UP (number, scale)</code>	Round half-up.
<code>ROUND_HALF_DOWN (number, scale)</code>	Round half-down.
<code>DECIMAL (number, scale)</code>	Decimal rounding.
<code>FLOOR (number)</code>	Floor (largest integer $\leq$ number).
<code>FLOOR_SCALED (number, scale)</code>	Scaled floor.
<code>CEIL (number)</code>	Ceiling (smallest integer $\geq$ number).
<code>CEILING_SCALED (number, scale)</code>	Scaled ceiling.
<code>MIN (value, ...)</code>	Minimum of values.
<code>MAX (value, ...)</code>	Maximum of values.
<code>SUM (list)</code>	Sum a numeric list.
<code>COUNT (list)</code>	Count items in a list.
<code>SQRT (number)</code>	Square root.
<code>LOG (number)</code>	Natural logarithm.
<code>EXP (number)</code>	Exponential (e^n).
<code>ODD (number)</code>	True when integer is odd.
<code>EVEN (number)</code>	True when integer is even.
<code>MODULO (dividend, divisor)</code>	Modulo remainder.
<code>POW (base, exponent)</code>	Power ($\text{base}^{\text{exponent}}$).
<code>PMT (...)</code> / <code>PMT2 (...)</code>	Loan payment calculation helpers.

7.4 List Functions

Function	Description
PRODUCT(list)	Product of all numbers in list.
MEAN(list)	Arithmetic mean.
MEDIAN(list)	Median value.
STDDEV(list)	Sample standard deviation.
MODE(list)	Most frequent values.
ALL(list)	True when all boolean items are true.
ANY(list)	True when at least one boolean item is true.
SUBLIST(list, start, len)	Extract a sublist slice.
INSERT_BEFORE(list, position, item)	Insert item before position.
REMOVE(list, position)	Remove item at position.
REVERSE(list)	Reverse list order.
INDEX_OF(list, item)	Positions of item in list.
LIST_CONTAINS(list, item)	True when list contains item.
APPEND(list, item)	Append item to end of list.
DISTINCT(list)	Remove duplicate values.
FLATTEN(list)	Flatten nested lists.
SORT(list)	Sort ascending.
SORT_DESC(list)	Sort descending.
UNION(list, list)	Union of two lists.
LIST_REPLACE(list, position, item)	Replace item at position.
ARR_LEN(list)	Length of list.

7.5 Date, Time, Duration, and Range Functions

Function	Description
DATE(...)	Build or parse a date value.
TIME(...)	Build or parse a time value.
DATETIME(...) / DATE_AND_TIME(...)	Build or parse a datetime value.
DURATION(text)	Parse ISO duration string.
YEARS_AND_MONTHS_DURATION(from, to)	Year-month duration between two dates.
DAYS_BETWEEN(from, to)	Day count between two dates.

Function	Description
<code>YEARS_BETWEEN(from, to)</code>	Year count between two dates.
<code>MONTHS_BETWEEN(from, to)</code>	Month count between two dates.
<code>DATE_ADD(date, duration)</code>	Add a duration to a date.
<code>DATE_FROM_PARTS(year, month, day)</code>	Build date from numeric parts.
<code>DAY_OF_WEEK(date)</code>	Day of week number (1=Monday).
<code>DAY_OF_YEAR(date)</code>	Day of year number.
<code>WEEK_OF_YEAR(date)</code>	ISO week number.
<code>LAST_DAY_OF_MONTH(date)</code>	Last calendar day of the month.
<code>MONTH_OF_YEAR(date)</code>	Month number (1–12).
<code>MONTH_OF_YEAR_NAME(date)</code>	Month name string.
<code>DAY_OF_WEEK_NAME(date)</code>	Day name string.
<code>IS_WEEKEND(date)</code>	True when date falls on Saturday or Sunday.
<code>RANGE(start, end)</code>	Build a range value.
<code>BEFORE / AFTER / MEETS / MET_BY / OVERLAPS / OVERLAPPED_BY / STARTS / STARTED_BY / DURING / INCLUDES / FINISHES / FINISHED_BY / COINCIDES / OVERLAPS_BEFORE / OVERLAPS_AFTER</code>	Range relation predicates.

7.6 JSON and Context Functions

Function	Description
<code>JSON_GET(json, path)</code>	Read value at JSON path.
<code>JSON_GET_STR(json, path)</code>	Read string at JSON path.
<code>JSON_GET_I64(json, path)</code>	Read integer at JSON path.
<code>JSON_GET_F64(json, path)</code>	Read float at JSON path.
<code>JSON_GET_BOOL(json, path)</code>	Read boolean at JSON path.
<code>TO_JSON(value)</code>	Convert value to JSON string.
<code>CONTEXT(key, value, ...)</code>	Build a context from key–value pairs.
<code>CONTEXT_PUT(context, key, value)</code>	Return context with one field set.
<code>CONTEXT_MERGE(context, context)</code>	Merge two context objects.
<code>PUT(context, key, value)</code>	Put a field into a context.

Function	Description
PUT_ALL(context, context)	Put all fields from one context into another.
GET_VALUE(context, key)	Read a value from a context by key.
GET_ENTRIES(context)	Return all entries of a context as a list.

7.7 Type Predicate Functions

Function	Description
IS_NUMBER(value)	True when value is a number.
IS_STRING(value)	True when value is a string.
IS_BOOL(value)	True when value is a boolean.
IS_LIST(value)	True when value is a list.
IS_OBJECT(value)	True when value is a context/object.
IS_RANGE(value)	True when value is a range.
IS_INSTANCE_OF(value, type)	True when value is an instance of named type.
ISBLANK(value)	True when value is null, empty string, or whitespace.
ISEMPTY(value)	True when list/string/context is empty.
IS_NULL_FN(value)	True when value is null.

8. Actions and Outputs

Each rule has a then list. Actions execute in order when the when expression is true.

8.1 SET_FIELD

Set a literal string output value:

```
- kind: SET_FIELD
  path: decision
  value: "APPROVE"
```

8.2 EVAL_SET_FIELD

Set a computed output value using a VDICL expression:

```
- kind: EVAL_SET_FIELD
  path: approvedLimit
  expr: monthlyIncome * 12
```

8.3 EMIT

Emit a structured finding for validation or compliance reporting:

```
- kind: EMIT
  severity: ERROR
  code: AGE_BELOW_MIN
  field: applicant.age
  msg: Applicant is below minimum age
```

Supported severities: PASS · INFO · WARN · ERROR · CRITICAL

8.4 ADD_SCORE

Add a numeric delta to the accumulated rule-pack score:

```
- kind: ADD_SCORE
  score_delta: 25
```

8.5 SET_DECISION

Set the rule-pack decision envelope directly:

```
- kind: SET_DECISION
  decision: REVIEW
```

Supported decisions: UNSET · ACCEPT · REJECT · REVIEW · CHALLENGE_MFA · CHALLENGE_APPROVAL

8.6 Control Actions

Action	Description
RETURN	Stop the current rule after prior actions have run.
ABORT	Stop all evaluation and mark result as aborted.
LOG	Emit a diagnostic log message (development use).

```
- kind: ABORT
  msg: Critical eligibility invariant failed
```

8.7 ASSIGN

Write a literal value to an internal (non-output) path:

```
- kind: ASSIGN
  path: tmp.reason
  value: "LOW_SCORE"
```

8.8 CALL_HELPER

Surface a helper request to the host application:

```
- kind: CALL_HELPER
  name: fetch_credit_bureau
  arg: applicant.nik
```

8.9 Action Order

Place finding and scoring actions before terminal control actions:

```
then:
- kind: EMIT
  severity: CRITICAL
  code: BLOCKED_CUSTOMER
  msg: Customer is blocked
- kind: SET_DECISION
  decision: REJECT
- kind: RETURN
```

9. Hit Policies

hit_policy controls which matching rules are selected and how their outputs combine.

9.1 Supported Policies

Policy	Behavior
FIRST	First matching rule wins. This is the default.
UNIQUE	At most one rule should match. Multiple matches are a logic error.
COLLECT	All matching rules execute and outputs may be aggregated.
PRIORITY	Rules ordered by priority integer; highest-priority match wins.
ANY	All matching rules must agree on each output field.
RULE_ORDER	All matching rules execute in document order.
OUTPUT_ORDER	DMN compatibility alias; runtime uses deterministic document order.

9.2 FIRST (default)

```
hit_policy: FIRST
rules:
  - id: r_preferred
    when: tier == "GOLD"
    then:
      - kind: SET_FIELD
        path: decision
        value: "APPROVE"
  - id: r_fallback
    when: true
    then:
      - kind: SET_FIELD
        path: decision
        value: "REVIEW"
```

9.3 COLLECT

Use COLLECT when multiple findings or aggregated outputs are expected:

```
hit_policy: COLLECT
aggregate_fn: LIST
```

Supported aggregate functions: LIST · SUM · MIN · MAX · COUNT

9.4 Output Conflict Policy

output_conflict_policy controls what happens when multiple matching rules write different values to the same output path:

```
output_conflict_policy: LWW      # last write wins (default)
output_conflict_policy: STRICT  # raises error on conflict
```

9.5 PRIORITY

priority is a signed integer on each rule. Lower numbers execute first within the group:

```
hit_policy: PRIORITY
rules:
  - id: r_high
    priority: 1
    when: risk == "LOW"
    then:
      - kind: SET_FIELD
        path: decision
        value: "APPROVE"
```

□ Use explicit priority only when the business rule genuinely depends on priority ordering. *FIRST* with document order is usually clearer.

10. DMN Unary Tests

VDICL tooling accepts DMN-style unary tests when importing decision tables from Camunda or other DMN sources. Unary tests are expressions written relative to an implicit left-hand side field.

Example: if the input expression is amount, the unary test:

```
< 1000
```

is translated as:

```
amount < 1000
```

10.1 Supported Unary Test Forms

Unary Test	Meaning
< 10	Less than 10.
<= 10	Less than or equal to 10.
> 0	Greater than 0.
>= 5	Greater than or equal to 5.
!= "USD"	Not equal to USD.
<> "USD"	Not equal (alias).
= "ACTIVE"	Equal to ACTIVE.
== "ACTIVE"	Equal to ACTIVE (alias).
[1..10]	Closed range: $1 \leq \text{value} \leq 10$.
]1..10[	Open range: $1 < \text{value} < 10$.
[1..10[	Closed lower, open upper: $1 \leq \text{value} < 10$.
]1..10]	Open lower, closed upper: $1 < \text{value} \leq 10$.
IN {"A", "B"}	Set membership.
NOT IN {"A", "B"}	Set exclusion.
-	Any value (wildcard match).

10.2 Authoring Equivalent

After import, prefer explicit full expressions in authored VDICL packs:

```
# Preferred authored form
when: amount >= 1 AND amount <= 10

# Imported from DMN
# [1..10] → amount >= 1 AND amount <= 10
```

□ *Unary tests are primarily an import and table-cell syntax. Rule-pack YAML should use full expressions wherever practical.*

11. Validation and Runtime

This section covers compiler and runtime behavior that authors must understand before publishing a rule pack.

11.1 Compile-Time Checks

The compiler parses each rule's when field as a VDICL expression and binds all field references against the schema dictionary:

```
|when: tier == "GOLD" AND approvedLimit > 10000
```

EVAL_SET_FIELD.expr is also parsed and bound:

```
|expr: monthlyIncome * 12
```

SET_FIELD.value is NOT parsed as an expression. It is a literal string:

```
|value: "APPROVE"
```

□ *Unknown field references in when or expr are compile errors, not warnings. Resolve all field paths against the schema before publishing.*

11.2 Runtime Result Envelope

Field	Description
decision	Decision text such as ACCEPT, REJECT, or REVIEW.
total_score	Accumulated score from ADD_SCORE actions.
findings_count	Number of EMIT findings or CALL_HELPER requests surfaced.
out	Materialized fields written by SET_FIELD or EVAL_SET_FIELD.

Diagnostic modes may include full findings, reasons, matched rule counts, and runtime details.

11.3 Null Behavior

Many builtins return null when an input is null. Write explicit fallbacks for business-critical comparisons:

```
|when: COALESCE(monthlyIncome, 0) >= 5000000
```

11.4 Disabled Rules

A rule defaults to enabled. Set enabled: false to skip it at runtime:

```
|enabled: false
```

11.5 Stable Fallback Rule

Every decision pack should end with an explicit fallback unless the business policy intentionally returns no decision when nothing matches:

```
- id: r_default_review
  description: Review all cases not matched by a specific rule
  when: true
  then:
    - kind: SET_FIELD
      path: decision
```

```
value: "REVIEW"
```

11.6 Recommended Validation Flow

- Parse YAML and validate rule-pack shape.
- Bind all expressions against the schema dictionary.
- Compile to a runtime pack.
- Run representative test cases covering each rule branch.
- Review diagnostics output before publishing.

Authoring tools and AI assistants should follow the same flow before offering an apply or publish action.

12. Complete Example

The following rule pack demonstrates a realistic loan eligibility decision using multiple rules, hit policy FIRST, and a stable fallback.

```
version: 1
decision_id: loan_eligibility
decision_name: Loan Eligibility Decision
hit_policy: FIRST
output_conflict_policy: LWW

schema:
  inputs:
    - name: age
      type: number
      expression: applicant.age
    - name: tier
      type: string
      expression: applicant.tier
    - name: monthlyIncome
      type: number
      expression: applicant.monthlyIncome
    - name: creditScore
      type: number
      expression: applicant.creditScore
    - name: blocked
      type: boolean
      expression: applicant.blocked
    - name: npwp
      type: string
      expression: applicant.npwp
  outputs:
    - name: decision
      type: string
    - name: approvedLimit
      type: number

rules:
  - id: r_blocked
    description: Reject blocked applicants immediately
    when: blocked == true
    then:
      - kind: EMIT
        severity: CRITICAL
        code: APPLICANT_BLOCKED
        field: applicant.blocked
        msg: Applicant is on the blocked list
      - kind: SET_FIELD
        path: decision
        value: "REJECT"
      - kind: RETURN

  - id: r_min_age
    description: Reject applicants below minimum age
    when: age < 21
```

```
then:
  - kind: EMIT
    severity: ERROR
    code: AGE_BELOW_MIN
    field: applicant.age
    msg: Applicant must be at least 21 years old
  - kind: SET_FIELD
    path: decision
    value: "REJECT"

- id: r_npwp_required
  description: Reject applicants without NPWP
  when: npwp IS NULL OR npwp == ""
  then:
    - kind: EMIT
      severity: ERROR
      code: NPWP_MISSING
      field: applicant.npwp
      msg: NPWP is required for loan application
    - kind: SET_FIELD
      path: decision
      value: "REJECT"

- id: r_gold_high_income
  description: Approve GOLD tier with strong income and credit
  when: tier == "GOLD" AND creditScore >= 750
        AND COALESCE(monthlyIncome, 0) >= 10000000
  then:
    - kind: SET_FIELD
      path: decision
      value: "APPROVE"
    - kind: EVAL_SET_FIELD
      path: approvedLimit
      expr: monthlyIncome * 24

- id: r_silver_standard
  description: Approve SILVER tier with acceptable credit
  when: tier IN {"SILVER", "GOLD"} AND creditScore >= 650
        AND COALESCE(monthlyIncome, 0) >= 5000000
  then:
    - kind: SET_FIELD
      path: decision
      value: "APPROVE"
    - kind: EVAL_SET_FIELD
      path: approvedLimit
      expr: monthlyIncome * 12

- id: r_default_review
  description: Route all remaining cases to manual review
  when: true
  then:
    - kind: SET_FIELD
      path: decision
      value: "REVIEW"
```

Example Result Envelope

For an applicant with tier=GOLD, creditScore=780, monthlyIncome=12000000, age=30, blocked=false, npwp=filled:

```
{
  "decision": "APPROVE",
  "total_score": 0,
  "findings_count": 0,
  "out": {
    "decision": "APPROVE",
    "approvedLimit": 288000000
  }
}
```